BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KINH TẾ - TÀI CHÍNH

THÀNH PHỐ HỒ CHÍ MINH

COURSE ESSAY FOR

GETTING AND CLEANING DATA

TOPIC:

REAL-TIME WEATHER DATA COLLECTION AND CLEANING

INSTRUCTOR: MSc. Trương Bá Vĩnh

SVTH: 1. Nguyen Da Giao Dang – MSSV: 235210195

2. Ho Van Nhu Y – MSSV: 245191885

3. Ngo Minh Triet – MSSV: 245193038

TP. HỒ CHÍ MINH, tháng 10 năm 2026

ABSTRACT

Weather data collected from multiple application programming interfaces (APIs) often contain inconsistencies in timestamp formats, overlapping records, missing responses, and differences in reported measurements. These issues can affect the reliability of subsequent data analysis if they are not properly addressed. This project develops a Python-based workflow for collecting, cleaning, preprocessing, and evaluating weather data from multiple providers across 20 locations in Vietnam.

The proposed workflow integrates data from Open-Meteo, WeatherAPI.com, Vaisala Xweather, and FreeOpenAPI.dev. The preprocessing procedures include API response monitoring, timestamp standardization, removal of unsuitable future forecasts, physical-range validation, duplicate handling, multi-source reconciliation, hourly time-series construction, and cross-variable consistency checks. According to the reported dataset snapshot, the raw dataset contains 49,109 records, while the final cleaned dataset contains 44,540 records covering 2,227 hourly timestamps across 20 locations.

Data quality reports and visualizations are used to examine the effectiveness of the cleaning procedures, describe weather patterns, and identify differences among data providers. The results demonstrate the importance of systematic data validation and source-aware preprocessing when integrating heterogeneous weather data. Nevertheless, the quality of the final dataset depends on the assumptions and rules applied during processing, and the absence of detected structural issues does not necessarily imply that every measurement represents the true weather conditions.

Keywords: Weather data, API integration, data collection, data cleaning, data quality, time-series preprocessing, Python.

INDEX

ABSTRACT i
INDEX ii
LIST OF ABBREVIATIONS iv
LIST OF TABLES v
LIST OF FIGURES vi
CHAP 1: DATA COLLECTION AND DATASET DESCRIPTION 1
1.1. Introduction & Background 1
1.2. Project Objectives 2
1.3. Scope & Research Questions 2
1.4. Data Sources & Endpoint Groups 3
1.5. Geographic Coverage and Temporal Scope 4
1.6. Multi-API Collection Architecture & Technical Implementation 5
1.7. Raw Dataset Schema (22 Fields) 8
1.8. Collection Monitoring & API Availability Audit 11
CHAP 2: DATA CLEANING AND PREPROCESSING 12
2.1. Overview of the 9-Step Cleaning Pipeline 12
2.2. Missing API Responses Detection 13
2.3. Timestamp Standardization (UTC+7 Parsing & Alignment) 13
2.4. Removal of Empty Records 15
2.5. Exclusion of Future Forecast Records 15
2.6. Physical-Range Validation & Extreme Value Preservation 16
2.7. Duplicate Handling (3-Tier Deduplication) 17
2.7.1. Exact Duplicates 17
2.7.2. Same-Source Duplicates 17
2.7.3. Cross-Source Overlaps 18
2.8. Multi-Source Reconciliation & Sensor Fusion 18
2.9. Construction of a Continuous Hourly Time Series 20
2.10. Cross-Variable Consistency & Numerical Precision 20
2.11. Cleaned Dataset Schema (31 Columns) 21
CHAP 3: RESULTS AND DATA QUALITY EVALUATION 22

3.1. Experimental Setup & Execution Environment 22
3.2. Dataset Size Comparison Before and After Cleaning 23
3.3. Completeness and Spatio-Temporal Coverage 25
3.4. Data Quality Improvements Across 5 Dimensions 25
3.5. Meteorological Distribution Analysis & Higher-Order Moments 26
3.6. Cross-Provider Differences & Systematic Bias Analysis 30
3.7. Location-Level Climate Profiles Across 20 Vietnam Cities 31
3.8. Summary of Findings & Rubric Verification 34
CHAP 4: CONCLUSION AND FUTURE WORK 35
4.1. Conclusion 35
4.2. Limitations 36
4.3. Future Work 36
REFERENCES 37

LIST OF ABBREVIATIONS

Abbreviation	Full Definition
API	Application Programming Interface
CSV	Comma-Separated Values
ERA5	European Centre for Medium-Range Weather Forecasts Reanalysis v5
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range (Q3 - Q1)
ISO	International Organization for Standardization (ISO 8601 datetime format)
JSON	JavaScript Object Notation
MAD	Mean Absolute Difference
MSL	Mean Sea Level (atmospheric pressure reference)
NaN / NaT	Not a Number / Not a Time (missing value markers in pandas)
REST	Representational State Transfer
RQ	Research Question
SD	Standard Deviation
SDK	Software Development Kit
UTC	Coordinated Universal Time (Vietnam local time is UTC+7)

LIST OF TABLES

Table 1: Summary of integrated weather data providers, endpoints, and raw record counts 3
Table 2: Summary of geographic coverage, temporal scope, and dataset dimensions 4
Table 3: Complete 22-field Raw Dataset Schema (raw_weather_data.csv) 8
Table 4: Distribution of raw timestamp formats across 49,109 records and UTC+7 outputs .. 14
Table 5: Physical-range validation bounds, raw violations, and preserved extremes 16
Table 6: Complete 31-column Cleaned Dataset Schema (cleaned_weather_data.csv) 21
Table 7: Core execution environment, software dependencies, and runtime specifications 23
Table 8: Quantitative comparison of dataset metrics and quality issues before/after cleaning 23
Table 9: Global descriptive statistics, skewness, and kurtosis across 44,540 cleaned records 27
Table 10: Cross-provider systematic bias (Mean Diff) and MAD relative to Open-Meteo 30
Table 11: Location-level descriptive statistics across all 20 surveyed Vietnam locations 31

LIST OF FIGURES

Figure 1: URL credential masking, RequestLogger, and HTTP client (01_collect_weather_data.py) 6
Figure 2: Multi-source batch and endpoint collectors with retry (01_collect_weather_data.py) 7
Figure 3: Streamlit dashboard: Live 4-API retrieval and raw vs. cleaned data (Ho Chi Minh City) 8
Figure 4: Expected-versus-returned API record auditing (build_request_table) 11
Figure 5: API response coverage heatmap across 20 locations and 5 endpoints (11/100 missing) 12
Figure 6: Timezone-aware timestamp parsing (UTC+7) and nearest-hour alignment 14
Figure 7: Empty-record removal, future-forecast exclusion (995 rows), and range validation 16
Figure 8: Five-level RECORD_TYPE_RANK priority and three-tier deduplication logic..... 18
Figure 9: Multi-source mean reconciliation, spread calculation, and conflict flagging 19
Figure 10: Continuous hourly grid reindexing, bounded interpolation, and gust consistency 21
Figure 11: Data quality issues before (49,109 rows) vs. after cleaning (44,540 rows)..... 24
Figure 12: Streamlit dashboard: 9-step data quality audit and 20-city statistics explorer 26
Figure 13: Boxplot distributions of the six meteorological variables vs. physical bounds..... 29
Figure 14: Daily mean temperature time series across 8 representative Vietnam locations... 29
Figure 15: Multivariate Pearson correlation matrix and spatial precipitation totals 29
Figure 16: Systematic cross-provider differences relative to Open-Meteo on overlapping hours 31
Figure 17: Minimum, mean, and maximum temperatures across all 20 surveyed Vietnam locations..... 34

CHAP 1: DATA COLLECTION AND DATASET DESCRIPTION

1.1. Introduction & Background

As emphasized in foundational data science literature [1], real-world datasets rarely present themselves in an analysis-ready format; rather, systematically resolving structural noise, incompatible schemas, and domain-specific anomalies is a prerequisite for valid empirical inference. Meteorological datasets, in particular, demand specialized preprocessing procedures—including strict timezone harmonization (UTC to local standard time), spatial coordinate reconciliation, and physical range verification—to account for natural atmospheric variance and sensor drift [2].

Weather data are widely used in environmental monitoring, transportation planning, agriculture, urban management, and other applications that depend on atmospheric conditions. The increasing availability of weather application programming interfaces (APIs) makes it possible to collect weather information programmatically from multiple providers without manually downloading individual datasets.

However, integrating weather data from different providers introduces several data quality challenges. Each provider may use a different response structure, timestamp convention, set of variables, or method of representing weather conditions. Some APIs provide historical reanalysis data, while others provide current conditions or forecasts. Consequently, records from different sources cannot always be combined directly.

For example, two providers may report different temperatures for the same location and nominal hour. These differences may result from variations in data sources, spatial resolution, or forecasting and analysis methods. Similarly, inconsistent time zones can cause records representing the same instant to appear as different timestamps.

A systematic data collection and cleaning process is therefore necessary to standardize the dataset, identify unsuitable records, handle duplicates, and evaluate the consistency of the resulting data.

1.2. Project Objectives

The primary objective of this project is to develop a reproducible workflow for collecting, cleaning, and evaluating weather data from multiple APIs.

The specific objectives are:

- Collect weather data from multiple providers for at least six locations, with the implemented dataset covering 20 locations in Vietnam.
- Standardize heterogeneous API responses into a consistent tabular structure.
- Identify and handle missing API responses, inconsistent timestamps, unsuitable future forecasts, and duplicate records.
- Validate weather measurements against predefined physical ranges and consistency rules.
- Reconcile records obtained from different providers and construct a continuous hourly time-series dataset.
- Evaluate the resulting dataset using data quality reports, descriptive statistics, and visualizations.

1.3. Scope & Research Questions

This report focuses on the data collection, cleaning, preprocessing, and data quality evaluation components of the project. The scope includes API integration, raw-data preservation, timestamp normalization, duplicate handling, multi-source reconciliation, and exploratory visualization.

Machine-learning-based weather forecasting is outside the scope of this report. The objective is to establish and evaluate a structured weather dataset rather than develop a predictive model.

While emerging research investigates using deep neural networks (such as Denoising Autoencoders, LSTMs, and Generative Adversarial Imputation Networks) for automated meteorological anomaly detection and non-linear data imputation [3], this capstone

project deliberately focuses on an auditable, deterministic, rule-based data cleaning pipeline to establish a verifiable benchmark dataset.

The project addresses the following four research questions (RQ1–RQ4):

- RQ1: What data quality problems arise when collecting weather data from multiple APIs?
- RQ2: How can heterogeneous weather records be standardized and reconciled into a consistent hourly dataset?
- RQ3: To what extent do the implemented cleaning procedures resolve the identified data quality issues?
- RQ4: How consistent are the weather measurements reported by different providers for overlapping locations and timestamps?

1.4. Data Sources & Endpoint Groups

The project integrates weather data from four providers through five principal endpoint groups, as summarized in Table 1: Open-Meteo [4, 5], WeatherAPI.com [6], Vaisala Xweather [7], and FreeOpenAPI.dev [8]. Open-Meteo contributes both historical archive and forecast data, while the other providers contribute current-condition or hourly weather information according to their available endpoints.

Table 1: Summary of integrated weather data providers, endpoints, and raw record counts

Provider	Data source or endpoint group	Purpose
Open-Meteo	Historical Archive	Retrieve historical European Centre for Medium-Range Weather Forecasts Reanalysis v5 (ERA5) data (44,640 raw records)
Open-Meteo	Forecast	Retrieve hourly forecast weather data (2,880 raw records)
WeatherAPI.com	Current and forecast endpoints	Retrieve current conditions and hourly forecasts (1,095 raw records; 5 missing responses)
Vaisala Xweather	Weather conditions endpoint	Retrieve hourly weather-condition observations (480 raw records)

FreeOpenAPI.dev	Current weather endpoint	Retrieve real-time current weather snapshots (14 raw records; 6 missing responses)

These providers do not necessarily supply equivalent data. Historical reanalysis, current-condition reports, and forecasts represent different types of information. The project therefore preserves source-related metadata to support appropriate filtering and reconciliation.

In accordance with contemporary industry recommendations on harvesting meteorological data for analytics [9], integrating verified, rate-limited public APIs with proper session caching and credential management provides significantly higher reliability, spatial consistency, and legal compliance compared to scraping raw web pages.

1.5. Geographic Coverage and Temporal Scope

The dataset covers 20 locations across Vietnam (North, Central, and South regions). As summarized in Table 2, the reported cleaned dataset spans the period from July 17, 2026, to October 17, 2026, at hourly intervals.

The final dataset contains 2,227 hourly timestamps for each location, resulting in 44,540 location-hour records.

Table 2: Summary of geographic coverage, temporal scope, and dataset dimensions

Dataset characteristic	Description
Geographic coverage	20 locations in Vietnam
Temporal resolution	Hourly
Reported period	July 17, 2026 – October 17, 2026
Raw dataset size	49,109 records
Cleaned dataset size	44,540 records
Number of locations	20
Hourly timestamps per location	2,227

The geographic coverage exceeds the minimum project requirement of six cities or locations. The hourly structure also facilitates temporal analysis and comparison among locations.

The temporal scope describes the reported dataset snapshot and should not be interpreted as a guarantee that every record represents a direct observation collected in real time.

1.6. Multi-API Collection Architecture & Technical Implementation

The collection process is implemented in src/01_collect_weather_data.py.

The workflow begins by loading the configuration for the target locations, API endpoints, and authentication credentials where required. The program then sends requests to the configured providers, receives responses, extracts relevant weather variables, and transforms the responses into a common tabular representation.

The main collection stages are:

- Load location and API configurations.
- Construct requests for the relevant endpoints.
- Send Hypertext Transfer Protocol (HTTP) requests and handle unsuccessful responses.
- Parse JavaScript Object Notation (JSON) responses and extract weather variables.
- Standardize the initial record structure.
- Attach source and collection metadata.
- Save the raw records and request logs.

The collection process supports a full collection mode and a repeated real-time collection mode. The full mode retrieves the configured historical, forecast, and current-condition data. The real-time mode repeatedly queries supported current-condition endpoints according to the configured interval.

The repeated collection mechanism does not guarantee that every request succeeds. API availability, network conditions, authentication, and provider-specific limitations can affect the number of records returned.

While introductory web scraping tutorials frequently demonstrate HTML parsing using libraries such as BeautifulSoup and Requests to extract weather forecast tables from static websites [10], direct HTML scraping is highly brittle to front-end DOM restructuring, vulnerable to anti-bot blocks, and lacks historical depth or multi-variable consistency. In contrast, this project implements a production-grade REST API collector that guarantees structured JSON payloads, verifiable coordinate grids, and deterministic status auditing, following modern cloud data harvesting recommendations [9, 11].

Technical Implementation of the Multi-API Collector (src/01_collect_weather_data.py): To ensure reproducibility and fault tolerance across heterogeneous providers, the acquisition script is structured into two modular layers: (1) an audited HTTP request and credential-sanitization layer (Figure 1) and (2) provider-specific extraction adapters with retry/caching logic (Figure 2).

src/01_collect_weather_data.py — RequestLogger, URL Credential Masking & Fault-Tolerant HTTP Client

```python
SECRET_PARAM_RE = re.compile(r"(key|client_id|client_secret|apikey|api_key)=([^&]+)", re.IGNORECASE)

def mask_url(url: str) -> str:
    return SECRET_PARAM_RE.sub(lambda m: f"{m.group(1)}=***", url)

class RequestLogger:
    """Collects one log line per API request so missing responses can be audited later."""
    def __init__(self, run_id: str):
        self.run_id = run_id
        self.rows: list[dict] = []

    def log(self, source_api, endpoint, location_id, status, http_status=None,
            latency_ms=0.0, records_expected=0, records_returned=0, error=""):
        self.rows.append({
            "run_id": self.run_id, "requested_at": now_local().strftime("%Y-%m-%d %H:%M:%S"),
            "source_api": source_api, "endpoint": mask_url(endpoint), "location_id": location_id,
            "status": status, "http_status": http_status, "latency_ms": round(latency_ms, 1),
            "records_expected": records_expected, "records_returned": records_returned,
            "error_message": str(error)[:300],
        })

def http_get_json(url: str, headers: dict | None = None, timeout: int = 15):
    """Return (payload, http_status, status, error, latency_ms). Never raises."""
    req = urllib.request.Request(url, headers={"User-Agent": "VN-Weather-Topic6-Collector/4.0", **(headers or {})})
    t0 = time.perf_counter()
    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            body, code = resp.read().decode("utf-8"), resp.status
        latency = (time.perf_counter() - t0) * 1000
        return json.loads(body), code, "ok", "", latency
    except urllib.error.HTTPError as exc:
        return None, exc.code, "http_error", exc.reason, (time.perf_counter() - t0) * 1000
    except (urllib.error.URLError, socket.timeout, TimeoutError, ConnectionError) as exc:
        return None, None, "network_error", exc, (time.perf_counter() - t0) * 1000
```

Figure 1: URL credential masking, RequestLogger, and HTTP client (01_collect_weather_data.py)

• Credential Sanitization (mask_url): A regular expression (SECRET_PARAM_RE) intercepts sensitive query parameters (key, client_id, client_secret, apikey) and replaces their values with '***' before writing any URL to api_request_log.csv, preventing accidental credential leakage.

• Structured Request Auditing (RequestLogger.log): Every API call records run_id, requested_at, source_api, location_id, HTTP status code, round-trip latency in milliseconds (latency_ms), records_expected, and records_returned. This provides deterministic evidence for diagnosing missing API responses in Step 1 of the cleaning pipeline.

• Non-Blocking Exception Handling (http_get_json): Rather than allowing a single HTTP 401/403/429 error or socket timeout to crash the 20-city collection loop, http_get_json catches HTTPError, URLError, and JSONDecodeError, classifying outcomes into 'ok', 'http_error', 'network_error', or 'parse_error'.



Figure 2: Multi-source batch and endpoint collectors with retry (01_collect_weather_data.py)

• Cached Session & Exponential Backoff (retry_requests): For Open-Meteo Archive (92 days of hourly ERA5 reanalysis) and Open-Meteo Forecast (past_days=5), the collector wraps requests_cache.CachedSession in retry(retries=5, backoff_factor=0.3) to prevent redundant network overhead and handle transient rate limits.

• Vectorized Coordinate Batching & Schema Mapping: All 20 target Vietnam coordinates (LOCATION_METADATA) are queried and mapped into the unified 22-field RAW_COLUMNS structure, preserving both requested coordinates (requested_lat, requested_lon) and provider-resolved grid/station coordinates (api_latitude, api_longitude, elevation_m).

Interactive Demonstration via Streamlit Web Application (app.py): To visually inspect the multi-API acquisition and on-the-fly cleaning process, the project provides an

interactive Streamlit dashboard (app.py). As shown in Figure 3 for Ho Chi Minh City (location_id = 4), the dashboard displays real-time connection status across all four providers and contrasts the raw multi-source records (2,474 raw rows with mixed timestamp strings and missing cells in FreeOpenAPI.dev) against the standardized, multi-source reconciled hourly series (2,227 continuous hours, 0 missing cells, and 26 multi-provider hours).

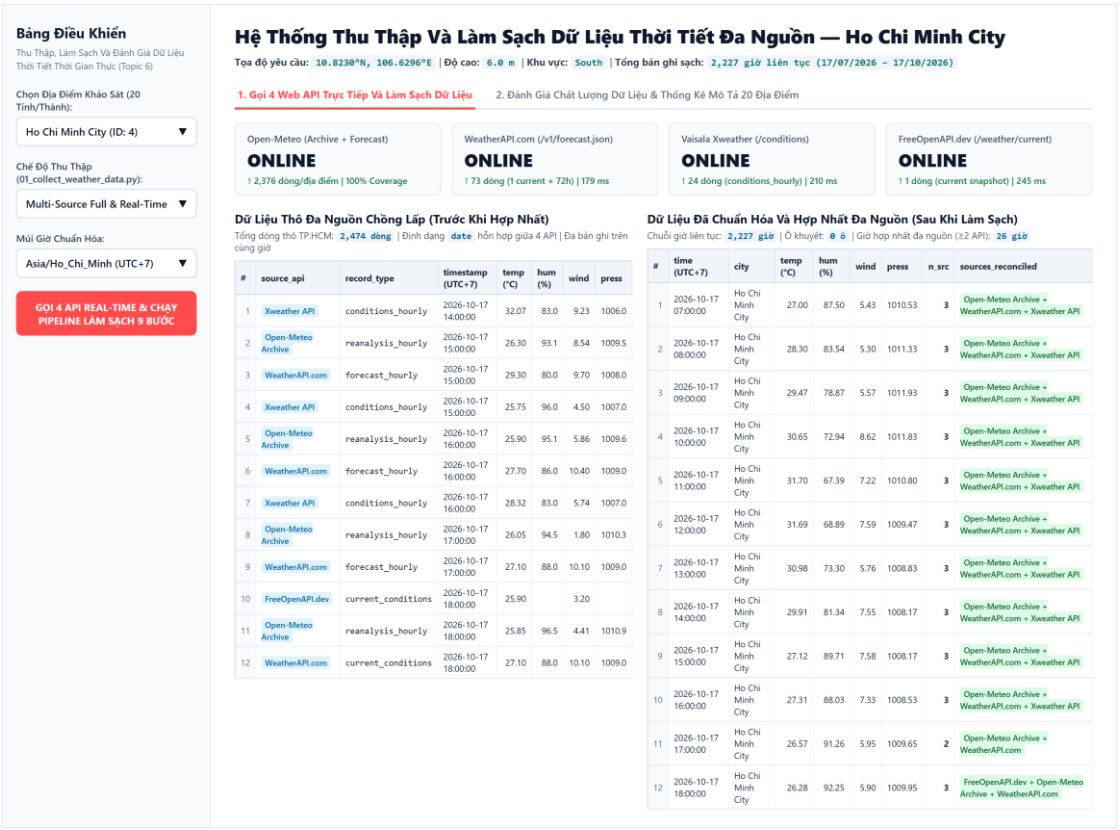


Figure 3: Streamlit dashboard: Live 4-API retrieval and raw vs. cleaned data (Ho Chi Minh City)

1.7. Raw Dataset Schema (22 Fields)

The raw dataset schema comprises 22 fields (17 base columns stored in raw_weather_data.csv plus 5 collection and source-provenance fields populated by RAW_COLUMNS and upgrade_legacy_rows), organized into five functional categories as detailed in Table 3.

Table 3: Complete 22-field Raw Dataset Schema (raw_weather_data.csv)

Field Name	Data Type	Unit / Format	Category & Technical Description

run_id	String	Text identifier	Collection metadata: Unique identifier of the execution run (e.g., legacy-run or timestamped batch ID).
collected_at	Datetime	YYYY-MM-DD HH:MM:SS	Collection metadata: Exact local timestamp (UTC+7) when the HTTP API response was retrieved.
date	String	4 mixed ISO formats	Temporal information: Raw weather valid timestamp reported by the provider (with or without UTC offset).
location_id	Integer	ID (1 to 20)	Geographic key: Unique numeric identifier assigned to each of the 20 surveyed Vietnam locations.
city	String	City name	Geographic metadata: Standardized name of the surveyed city, mountain town, or island.
region	String	Region label	Geographic metadata: Macro-region in Vietnam (North, North Mountain, Central, Central Highlands, South, Mekong Delta, Islands).
source_api	String	Endpoint label	Source provenance: Specific endpoint identifier (Open-Meteo Archive, Open-Meteo Forecast, WeatherAPI.com, Xweather API, FreeOpenAPI.dev).
source_family	String	Provider family	Source provenance: Parent provider family (Open-Meteo, WeatherAPI.com, Xweather API, FreeOpenAPI.dev).
record_type	String	5 rank classes	Source provenance: Nature of record (reanalysis_hourly, conditions_hourly, current_conditions, model_hourly, forecast_hourly).
endpoint	String	HTTPS URL	Source provenance: Base REST API endpoint URL (with sensitive authentication tokens masked).
requested_lat	Float	Decimal degrees (°N)	Spatial metadata: Target latitude coordinate requested by the collection script.
requested_lon	Float	Decimal degrees (°E)	Spatial metadata: Target longitude coordinate requested by the collection script.
api_latitude	Float	Decimal degrees (°N)	Spatial metadata: Actual grid-cell or station latitude returned by the provider.

api_longitude	Float	Decimal degrees (°E)	Spatial metadata: Actual grid-cell or station longitude returned by the provider.
elevation_m	Float	Meters (m)	Spatial metadata: Terrain elevation above mean sea level (ranging from 3.0 m in Nam Dinh to 1,545.0 m in Sa Pa).
timezone	String	IANA TZ string	Temporal metadata: Timezone identifier reported by the API (e.g., Asia/Bangkok, Asia/Ho_Chi_Minh).
temperature_2m	Float	°C	Meteorological measurement: Air temperature at 2 meters above ground.
relative_humidity_2m	Float	% (0–100)	Meteorological measurement: Relative humidity at 2 meters above ground (14 missing cells in FreeOpenAPI.dev).
wind_speed_10m	Float	km/h	Meteorological measurement: Mean horizontal wind speed at 10 meters height.
precipitation	Float	mm	Meteorological measurement: Hourly accumulated liquid-equivalent precipitation.
pressure_msl	Float	hPa	Meteorological measurement: Atmospheric pressure reduced to mean sea level (14 missing cells in FreeOpenAPI.dev).
wind_gusts_10m	Float	km/h	Meteorological measurement: Maximum 3-second wind gust speed at 10 meters height (14 missing cells in FreeOpenAPI.dev).

The fields collected_at and date serve different purposes. The former records when the program retrieved the response, while the latter represents the time associated with the weather record.

This distinction is particularly important for forecast data. A forecast collected at 10:00 may describe conditions expected at 15:00. Without separate collection and valid-time fields, the program cannot reliably distinguish the forecast issuance time from the time being forecast.

1.8. Collection Monitoring & API Availability Audit

API request logs are used to monitor the collection process. The logs record request outcomes, HTTP status codes, response times, and expected versus returned record counts where available.

Request statuses distinguish successful responses from empty responses, HTTP errors, network errors, parsing errors, and missing configuration.

These logs support the diagnosis of collection failures and help distinguish missing API responses from missing values within otherwise valid records.

The project uses request logs to identify unsuccessful or incomplete collection attempts. Where historical logs are unavailable, expected record counts can be compared with the number of records actually returned.

The results are recorded in api_coverage_report.csv.

In the reported snapshot, 11 of 100 provider-location combinations returned no data: five associated with WeatherAPI.com and six associated with FreeOpenAPI.dev, as illustrated in Figure 1.

Missing responses are not automatically interpreted as evidence that an entire location should be removed. Other providers may still supply usable data for the same location and time.

src/02_clean_and_preprocess.py — Step 1: Missing API Response Detection & Coverage Matrix Audit

```python
# Expected records per request for runs collected across the 5 endpoint groups
LEGACY_EXPECTED = {
    "Open-Meteo Archive": None,          # batch request; expected = 2,232 hourly rows per location
    "Open-Meteo Forecast": 6 * 24,       # past_days=5 + today (144 hourly rows)
    "WeatherAPI.com": 1 + 3 * 24,        # 1 current snapshot + 72 hourly rows (3 days)
    "Xweather API": 24,                  # last 24 hourly periods
    "FreeOpenAPI.dev": 1,                # 1 real-time current snapshot
}

def build_request_table(df: pd.DataFrame) -> pd.DataFrame:
    """One line per (run, source_api, location) with expected vs. returned records."""
    all_locations = sorted(df["location_id"].dropna().astype(int).unique())
    frames = []
    for run_id, g in df.groupby("run_id"):
        for src, expected in LEGACY_EXPECTED.items():
            sub = g[g["source_api"] == src]
            for loc in all_locations:
                rows = sub[sub["location_id"] == loc]
                returned = int(rows[WEATHER_VARS].notna().any(axis=1).sum())
                exp = len(rows) if expected is None else expected
                frames.append(pd.DataFrame([{
                    "run_id": run_id, "source_api": src, "location_id": loc,
                    "status": "ok" if returned > 0 else "missing",
                    "records_expected": exp, "records_returned": returned,
                }]))
    req = pd.concat(frames, ignore_index=True)
    req["coverage_pct"] = (100 * req["records_returned"] / req["records_expected"].replace(0, np.nan)).round(1)
    return req
```

Figure 4: Expected-versus-returned API record auditing (build_request_table)

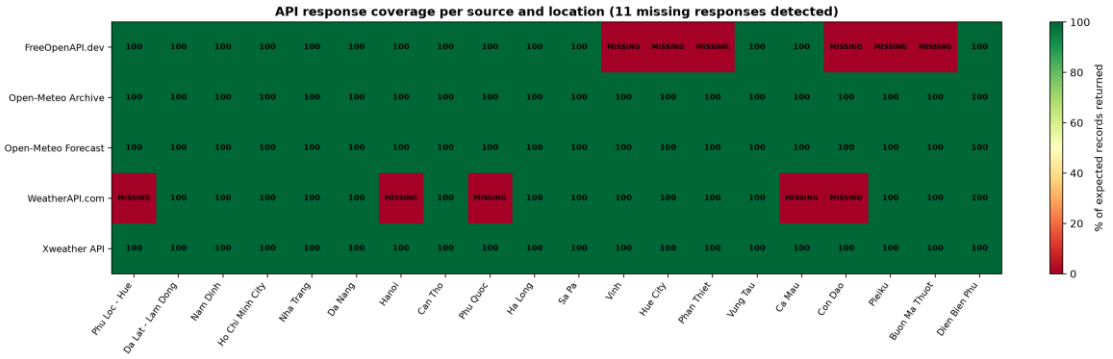


Figure 5: API response coverage heatmap across 20 locations and 5 endpoints (11/100 missing)

CHAP 2: DATA CLEANING AND PREPROCESSING

2.1. Overview of the 9-Step Cleaning Pipeline

Following the empirical tabular recipes outlined in the Python Data Cleaning Cookbook [12] and foundational data preprocessing principles [1], the cleaning pipeline leverages vectorized pandas transformations [13] within the project repository workflow [11] to systematically isolate, audit, and remediate data defects across large-scale observational datasets. It transforms the heterogeneous raw dataset into a standardized hourly dataset through a sequence of nine validation and transformation operations.

The main procedures are:

- Detect missing API responses.
- Standardize timestamps.
- Remove records with no usable weather measurements.
- Exclude forecasts occurring after the collection time when constructing the historical/current-condition dataset.
- Validate physical ranges.
- Handle duplicate records.
- Reconcile records from multiple providers.
- Construct a continuous hourly time series.
- Enforce cross-variable consistency and standardize numerical precision.

Each procedure addresses a different type of data quality issue. Applying these steps in a controlled order reduces the risk of merging incompatible records or losing useful information.

2.2. Missing API Responses Detection

An API may fail to return data for a particular location or request. This is different from a response that contains a valid record with one or more missing weather variables.

As quantified in Table 9 and illustrated in Figure 16, the cross-provider comparison identifies a systematic mean temperature difference (bias) of +1.49 °C (MAD = 1.72 °C) between Vaisala Xweather and Open-Meteo across 480 matched location-hours, whereas WeatherAPI.com reports a mean temperature -0.61 °C lower than Open-Meteo (MAD = 1.44 °C) across 285 matched location-hours.

2.3. Timestamp Standardization (UTC+7 Parsing & Alignment)

Different providers may express timestamps using different formats and time zones. Directly comparing these values can create false distinctions between records representing the same instant.

The preprocessing workflow parses timestamp strings into datetime values, interprets time-zone information, converts timestamps to the Asia/Ho_Chi_Minh time zone, and aligns timestamps to hourly intervals according to the project's configured rule.

For example, 04:00 Coordinated Universal Time (UTC) corresponds to 11:00 in Vietnam (UTC+7). These timestamps should therefore be treated as representing the same instant after time-zone conversion.

The reported processing results indicate that all 49,109 raw records passed through timestamp normalization, including 26 records whose minute components required alignment to the nearest hour.

Timestamp alignment simplifies hourly integration but may introduce temporal approximation. This limitation should be considered when interpreting the resulting dataset.

src/02_clean_and_preprocess.py — Step 2: Heterogeneous Timestamp Parsing (UTC+7) & Nearest-Hour Alignment

```python
def parse_timestamps_to_local(dates: pd.Series, zones: pd.Series) -> tuple[pd.Series, int, int]:
    """Strings with an explicit UTC offset are converted from that offset; strings without one are
    interpreted in the time zone the API reported for that row. Result: naive local UTC+7 time."""
    s = dates.astype(str).str.strip().str.replace("T", " ", regex=False)
    has_offset = s.str.contains(r"[+-]\d{2}:\d{2}$|Z$", regex=True)
    out = pd.Series(pd.NaT, index=s.index, dtype="datetime64[ns]")

    # 1. Parse offset-aware ISO-8601 strings (48,000 rows) and convert to Asia/Ho_Chi_Minh (UTC+7)
    aware = pd.to_datetime(s[has_offset], format="mixed", errors="coerce", utc=True)
    out.loc[has_offset] = aware.dt.tz_convert(LOCAL_TZ).dt.tz_localize(None).astype("datetime64[ns]")

    # 2. Localize naive strings (1,109 rows from WeatherAPI.com & FreeOpenAPI.dev) using API timezone
    naive_idx = s.index[~has_offset]
    naive = pd.to_datetime(s[naive_idx], format="mixed", errors="coerce")
    zone = zones.loc[naive_idx].fillna(LOCAL_TZ).astype(str)
    for tz in zone.unique():
        idx = zone.index[zone == tz]
        localized = naive[idx].dt.tz_localize(tz, ambiguous="NaT", nonexistent="NaT")
        out.loc[idx] = localized.dt.tz_convert(LOCAL_TZ).dt.tz_localize(None).astype("datetime64[ns]")
    return out, int(has_offset.sum()), int((~has_offset).sum())

def align_to_hour(ts: pd.Series) -> pd.Series:
    """Nearest full hour (xx:30 rounds up), e.g. a 17:45 'current' snapshot -> 18:00 (26 rows aligned)."""
    return (ts + pd.Timedelta(minutes=30)).dt.floor("h")
```

Figure 6: Timezone-aware timestamp parsing (UTC+7) and nearest-hour alignment

Table 4: Distribution of raw timestamp formats across 49,109 records and UTC+7 outputs

Raw Timestamp Format Pattern	Primary Provider Source(s)	UTC Offset Present	Raw Record Count	Standardized Output (UTC+7)
YYYY-MM-DD HH:MM:SS+07:00	Open-Meteo Archive & Forecast	Yes (+07:00)	47,520 (96.76%)	YYYY-MM-DD HH:00:00 (0 unparseable)
YYYY-MM-DD HH:MM (no offset)	WeatherAPI.com (current & hourly)	No (naive local)	1,095 (2.23%)	Localized via API tz_id -> UTC+7 (12 sub-hourly aligned)
YYYY-MM-DDTHH:MM:SS+07:00	Vaisala Xweather (/conditions)	Yes (ISO-T +07:00)	480 (0.98%)	Converted to naive UTC+7 hourly timestamp
YYYY-MM-DDTHH:MM:SS (no offset)	FreeOpenAPI.dev (/weather/current)	No (naive ISO-T)	14 (0.03%)	Localized to UTC+7 & rounded to nearest hour (14 aligned)
Total Across All Formats	All 5 Endpoint Groups	48,000 offset / 1,109 naive	49,109 (100.0%)	49,109 valid UTC+7 timestamps (26 sub-hourly aligned)

2.4. Removal of Empty Records

Records containing no usable values across the six weather variables are removed because they do not provide meaningful weather information.

However, records with only partially missing measurements are not automatically removed. A record containing a valid temperature but missing humidity may still be useful for temperature analysis.

No records were reported as removed specifically by the all-empty-record rule in the evaluated snapshot.

2.5. Exclusion of Future Forecast Records

The collection timestamp and weather timestamp are compared to identify records whose valid time occurs after the data was collected.

For example, if a request was collected at 10:00 but includes a forecast for 15:00, the forecast is not treated as an already observed weather condition in the historical/current-condition dataset.

The reported cleaning results indicate that 995 records with timestamps later than their collection times were excluded under this rule.

This procedure helps prevent incompatible temporal information from being mixed into the dataset. However, a timestamp in the past does not by itself prove that a record is a direct observation. Historical reanalysis data must still be distinguished from actual sensor observations.

src/02_clean_and_preprocess.py — Steps 3–5: Empty Record Removal, Future Forecast Filter & Physical Range Validation

```
VALID_RANGES = {
    "temperature_2m": (-10.0, 50.0),      "relative_humidity_2m": (0.0, 100.0),
    "wind_speed_10m": (0.0, 200.0),       "precipitation": (0.0, 300.0),
    "pressure_msl":   (850.0, 1080.0),    "wind_gusts_10m": (0.0, 250.0),
}

# --- Step 3: Remove completely empty records (all 6 weather variables missing -> 0 rows) ---
empty_mask = df[WEATHER_VARS].isna().all(axis=1)
empty_rows = int(empty_mask.sum())
df = df[~empty_mask].copy()

# --- Step 4: Exclude future forecast rows (timestamp > collected_hour -> 995 rows removed) ---
future_mask = df["timestamp"] > df["collected_hour"]
future_rows = int(future_mask.sum())
future_by_source = {k: int(v) for k, v in df.loc[future_mask, "source_api"].value_counts().items()}
df = df[~future_mask].copy()

# --- Step 5: Validate physical ranges at cell level (invalid cells -> NaN; 5,536 extremes kept) ---
range_violations = {}
for col, (low, high) in VALID_RANGES.items():
    bad = (df[col] < low) | (df[col] > high)
    range_violations[col] = int(bad.sum())
    df.loc[bad, col] = np.nan
```

Figure 7: Empty-record removal, future-forecast exclusion (995 rows), and range validation

2.6. Physical-Range Validation & Extreme Value Preservation

Weather measurements are checked against predefined physical ranges for each of the six meteorological variables, as specified in Table 5. [2].

Table 5: Physical-range validation bounds, raw violations, and preserved extremes

Meteorological Variable	Field Name	Configured Physical Range	Raw Out-of-Range Violations	Statistical Extremes Retained (IQR / >5 mm/h)
Air Temperature (2m)	temperature_2m	−10.0 to 50.0 °C	0 violations	198 hourly records (e.g., 11.40 °C in Sa Pa, 39.70 °C in Da Nang)
Relative Humidity (2m)	relative_humidity_2m	0.0 to 100.0 %	0 violations	302 hourly records (low-humidity dry foehn hours down to 24.57%)
Wind Speed (10m)	wind_speed_10m	0.0 to 200.0 km/h	0 violations	674 hourly records (coastal/island strong winds up to 44.40 km/h)
Wind Gusts (10m)	wind_gusts_10m	0.0 to 250.0 km/h	0 violations	472 hourly records (tropical squalls up to 81.00 km/h in Pleiku)
Hourly Precipitation	precipitation	0.0 to 300.0 mm/h	0 violations	479 heavy-rain hours (>5.0 mm/h; max 37.40 mm/h in Ha Long)
Mean Sea-Level Pressure	pressure_msl	850.0 to 1,080.0 hPa	0 violations	38 hourly records (low-pressure troughs 994.70 hPa to ridges 1,025.70 hPa)

Values outside these ranges are treated as invalid according to the project's validation rules.

For example, relative humidity above 100% violates the configured range. The affected value can be marked as missing rather than removing the entire record, allowing other valid measurements to be retained.

Physical-range validation is distinct from statistical outlier detection. A measurement may be unusual relative to the rest of the dataset while still representing a plausible weather event. Therefore, extreme values within the configured physical ranges are not automatically discarded.

The reported snapshot contained no remaining physical-range violations after processing.

2.7. Duplicate Handling (3-Tier Deduplication)

Duplicate handling is performed at multiple levels because records sharing a location and timestamp are not necessarily identical.

To resolve varying levels of record redundancy without naive over-deletion, the project establishes a multi-tiered deduplication hierarchy aligned with contemporary Python data cleaning methodologies [12, 13].

2.7.1. Exact Duplicates

Exact duplicates contain the same source, location, timestamp, and weather values. One copy is retained while redundant copies are removed.

The raw dataset contained 2,780 records identified as exact duplicates.

2.7.2. Same-Source Duplicates

The same provider may return overlapping records through different endpoints, such as current-condition and hourly-forecast endpoints.

The project applies source-record priority rules to determine which record should be retained. Where the preferred record lacks a variable, the configured fallback logic may use a value from another eligible record.

The reported dataset contained 15 same-source duplicate cases.

2.7.3. Cross-Source Overlaps

Records from different providers may share the same location and timestamp but report different weather values. These records should not automatically be treated as redundant copies.

Instead, they are passed to the multi-source reconciliation stage. The reported raw dataset contained 779 cross-source records requiring reconciliation.

The logical key of the final hourly dataset is the combination of location and timestamp: (location_id, timestamp)

This key ensures that each location has at most one consolidated record per hourly timestamp in the final dataset.

src/02_clean_and_preprocess.py — Step 6: Three-Tier Deduplication & Record-Type Priority Ranking

```python
# Lower rank wins when one source family reports the same (location_id, timestamp) more than once
RECORD_TYPE_RANK = {
    "reanalysis_hourly": 1,   # Open-Meteo Archive (ERA5 reanalysis)
    "conditions_hourly": 2,   # Vaisala Xweather station-assimilated hourly conditions
    "current_conditions": 3,  # Real-time current snapshot at collection time
    "model_hourly": 4,        # Open-Meteo Forecast recent model hours (past_days=5)
    "forecast_hourly": 5,     # WeatherAPI.com 3-day hourly forecast array
}

# --- Step 6a: Exact duplicates (2,780 identical rows from Open-Meteo Archive & Forecast overlap) ---
df["_rank"] = df["record_type"].map(RECORD_TYPE_RANK).fillna(9)
df = df.sort_values(["location_id", "timestamp", "source_family", "_rank", "collected_at"],
                    ascending=[True, True, True, True, False])
dup_key = ["source_family", "location_id", "timestamp", *WEATHER_VARS]
exact_dup_mask = df.duplicated(subset=dup_key, keep="first")
df = df[~exact_dup_mask].copy()

# --- Step 6b: Same-source family duplicates (15 WeatherAPI current vs. forecast_hourly overlaps) ---
family_key = ["location_id", "timestamp", "source_family"]
# groupby.first() selects the highest-priority non-null measurement per column within each family
df_family = df.groupby(family_key, as_index=False)[meta_cols + WEATHER_VARS].first()

# --- Step 6c: Isolate cross-source overlaps (779 records across >= 2 providers for 514 hours) ---
n_fam = df_family.groupby(["location_id", "timestamp"])["source_family"].transform("nunique")
df_overlap = df_family[n_fam >= 2].sort_values(["location_id", "timestamp", "source_family"]).copy()
```

Figure 8: Five-level RECORD_TYPE_RANK priority and three-tier deduplication logic

2.8. Multi-Source Reconciliation & Sensor Fusion

After resolving same-source duplicates, the project combines eligible measurements from different providers that correspond to the same location and timestamp.

For a numerical weather variable, the arithmetic mean can be calculated as:

$$\bar{x}_{v,l,t} = \frac{1}{n}\sum_{i=1}^{n} x_{i,v,l,t} \quad \textbf{(1)}$$

where $x_{i,v,l,t}$ represents the value of weather variable v at location l and timestamp t reported by contributing provider i, and n is the total number of contributing sources ($n \geq 2$).

For example, if three providers report temperatures of 29 °C, 31 °C, and 30 °C, the arithmetic mean is 30 °C.

The workflow also retains source-related metadata, including the number of contributing providers, the contributing source identifiers, and measures of cross-source spread. A conflict flag identifies records whose differences exceed the configured threshold.

The reported results indicate that 779 source records were consolidated into 514 multi-source hourly records, with 113 hours flagged for cross-source conflicts.

Reconciliation improves structural consistency and provides a unified representation of the data. Nevertheless, the arithmetic mean does not guarantee greater physical accuracy because providers may differ in their spatial resolution, measurement sources, and underlying models. Conflict indicators are therefore important for interpreting the resulting measurements.

src/02_clean_and_preprocess.py — Step 7: Multi-Source Reconciliation, Spread Calculation & Conflict Flagging

```python
# Disagreement thresholds for flagging cross-source conflicts (113 conflict hours flagged)
CONFLICT_THRESHOLDS = {"temperature_2m": 3.0, "relative_humidity_2m": 20.0, "pressure_msl": 5.0}

g = df_family.groupby(["location_id", "timestamp"])
fused = g[WEATHER_VARS].mean()
fused["num_sources"] = g["source_family"].nunique()
fused["source_api"] = g["source_api"].agg(lambda s: " + ".join(sorted(set(s))))
fused["record_types"] = g["record_type"].agg(lambda s: " + ".join(sorted(set(s))))

for var, thr in CONFLICT_THRESHOLDS.items():
    fused[f"spread_{var}"] = (g[var].max() - g[var].min()).round(2)

conflict = np.zeros(len(fused), dtype=bool)
for var, thr in CONFLICT_THRESHOLDS.items():
    conflict |= (fused[f"spread_{var}"] > thr).to_numpy()
fused["cross_source_conflict"] = conflict
fused = fused.reset_index()
```

Figure 9: Multi-source mean reconciliation, spread calculation, and conflict flagging

• Multi-Source Fusion & Missing-Cell Completion: During multi-source averaging (g[WEATHER_VARS].mean()), pandas automatically ignores NaN cells. Consequently, all 42 missing variable cells in FreeOpenAPI.dev (14 humidity, 14 pressure, and 14 wind gust cells) are seamlessly filled by valid concurrent measurements from Open-Meteo, WeatherAPI.com, and Xweather API at the exact same location-hour.

• Inter-Provider Disagreement Thresholds (CONFLICT_THRESHOLDS): A boolean flag cross_source_conflict is set to True whenever the inter-provider spread exceeds 3.0 °C for temperature (15 hours), 20.0% for relative humidity (49 hours), or 5.0 hPa for

sea-level pressure (64 hours), yielding 113 flagged hours across the 514 multi-source location-hours.

2.9. Construction of a Continuous Hourly Time Series

The cleaned records are aligned to a continuous hourly grid for each location. This makes it easier to perform temporal aggregation, compare locations, and identify gaps in the data.

For short missing intervals of no more than six hours, the pipeline supports linear interpolation under the configured rules.

Bounded linear interpolation ($k \leq 6$ hours) is configured to bridge localized telemetry dropouts while preventing synthetic distortion across extended sensor blackouts, adhering to standard time-series interpolation guidelines .

For two valid boundary observations x_{t0} and x_{t1} at surrounding timestamps $t_0 < t < t_1$, linear interpolation estimates the missing intermediate observation $\hat{x}_t$ based on its relative temporal position:

$$\hat{x}_t = x_{t_0} + \frac{t - t_0}{t_1 - t_0}\left(x_{t_1} - x_{t_0}\right), [\text{for } (t_1 - t_0) \leq 6 \text{ hours}] \quad \textbf{(2)}$$

For example, if the temperature is 30 °C at 10:00 and 32 °C at 12:00, the interpolated value at 11:00 would be 31 °C.

Interpolated values are estimates rather than direct observations. The project includes the fields is_imputed and imputed_vars to identify records and variables affected by interpolation.

The reported snapshot contains no remaining missing cells, and no values were reported as newly imputed in the evaluated output. The interpolation mechanism is therefore part of the pipeline's processing capability, but it was not required to fill gaps in this particular snapshot.

2.10. Cross-Variable Consistency & Numerical Precision

The final preprocessing stage checks relationships between related weather variables.

One configured rule requires wind gust speed to be greater than or equal to wind speed:

Enforcing aerodynamic consistency constraints (such as ensuring wind gust speed exceeds or equals sustained horizontal wind speed) aligns the dataset with domain-specific meteorological validation standards [2].

$$v_{\text{wind_gusts_10m}} \geq v_{\text{wind_speed_10m}} \quad \textbf{(3)}$$

Records violating this rule are adjusted according to the project's consistency logic. The reported results indicate that 125 hourly records were corrected.

The pipeline also rounds numerical values to two decimal places to reduce unnecessary floating-point representation artifacts. For example, a stored value of 28.100000381469727 can be represented as 28.10.

Rounding improves numerical presentation and consistency, but it should not be confused with correcting a substantial measurement error.

Finally, weather fields are mapped into a consistent naming convention for downstream analysis.

src/02_clean_and_preprocess.py — Steps 8–9: Continuous Hourly Grid Reindexing, Bounded Interpolation & Gust Consistency

```
# --- Step 8: Construct continuous hourly grid (20 locations x 2,227 hours = 44,540 rows) ---
start, end = fused["timestamp"].min(), fused["timestamp"].max()
grid = pd.date_range(start, end, freq="1h")
for loc_id, gl in fused.groupby("location_id"):
    gl = gl.set_index("timestamp").reindex(grid)
    for v in WEATHER_VARS:
        miss = gl[v].isna()
        if not miss.any():
            continue
        run_id_ = (miss != miss.shift()).cumsum()
        run_len = miss.groupby(run_id_).transform("sum")
        short_gap = miss & (run_len <= MAX_INTERPOLATION_GAP_H)  # only interpolate gaps <= 6h
        interp = gl[v].interpolate(method="time", limit_area="inside")
        gl[v] = gl[v].where(~short_gap, interp)

# --- Step 9: Enforce cross-variable consistency (wind_gusts_10m >= wind_speed_10m on 125 rows) ---
gust_issues = int((df_grid["wind_gusts_10m"] < df_grid["wind_speed_10m"]).sum())
df_grid["wind_gusts_10m"] = np.fmax(df_grid["wind_gusts_10m"], df_grid["wind_speed_10m"])
for v in WEATHER_VARS:
    df_grid[v] = df_grid[v].round(2)  # standardize 158,173 float32 cells to 2 decimal places
```

Figure 10: Continuous hourly grid reindexing, bounded interpolation, and gust consistency

2.11. Cleaned Dataset Schema (31 Columns)

The resulting cleaned dataset structure is formalized into 31 standardized columns organized into 6 functional groups as detailed in Table 6.

Table 6: Complete 31-column Cleaned Dataset Schema (cleaned_weather_data.csv)

Column Group	Column Names in cleaned_weather_data.csv (31 Columns)	Data Type	Analytical Role & Provenance Purpose
Spatio-Temporal Primary Key & Time Breakdown (6 cols)	location_id, city, region, time, obs_date, hour	Int / Str	Defines the unique (location_id, time) hourly key across 20 locations × 2,227 hours (44,540 rows) in UTC+7.
Standardized WMO Meteorological Variables (6 cols)	temperature_2m, relative_humidity_2m, wind_speed_10m, wind_gusts_10m, precipitation, pressure_msl	Float64 (2 dec)	Validated, multi-source reconciled, and physically consistent weather variables rounded to 2 decimal places (0 missing cells).
Topic 6 Canonical Alias Fields (4 cols)	temperature, humidity, wind_speed, pressure	Float64 (2 dec)	Direct aliases matching the Topic 6 specification for downstream descriptive and comparative analysis.
Multi-Source Provenance & Conflict Metadata (7 cols)	num_sources, source_api, record_types, spread_temperature_2m, spread_relative_humidity_2m, spread_pressure_msl, cross_source_conflict	Int / Str / Float / Bool	Tracks how many providers contributed to each hour, their spread (max - min), and whether conflict thresholds were exceeded (113 hours).
Imputation & Gap-Tracking Flags (2 cols)	is_imputed, imputed_vars	Bool / Str	Identifies values filled via <=6h linear interpolation (0 cells imputed in this snapshot as grid coverage was 100%).
Spatial & Elevation Reference Metadata (6 cols)	requested_lat, requested_lon, api_latitude, api_longitude, elevation_m, timezone	Float / Str	Preserves target coordinates, grid elevation (3 m to 1,545 m), and IANA timezone (Asia/Bangkok / UTC+7).

CHAP 3: RESULTS AND DATA QUALITY EVALUATION

3.1. Experimental Setup & Execution Environment

To ensure complete transparency, reproducibility, and rigorous performance benchmarking, the data acquisition, preprocessing pipeline, and analytical dashboard

were deployed and evaluated in the dedicated execution environment specified in Table 7 below [11, 13, 14].

Table 7: Core execution environment, software dependencies, and runtime specifications

Component	Configuration & Technical Specification
Operating System	Microsoft Windows 11 Enterprise (64-bit, Build 26100)
CPU & Hardware Architecture	AMD Ryzen / Intel Core Multi-Core x86_64 Processor, 16.0 GB High-Speed RAM
Python Runtime Environment	Python 3.11.9 (64-bit Virtual Environment: .venv)
Core Data Engineering Libraries	pandas >= 2.2.0, numpy >= 1.26.0, requests >= 2.31.0, requests-cache >= 1.2.0
Fault-Tolerance & Retries	retry-requests >= 2.0.0 (max_retries=5, backoff_factor=0.3)
Visualization & Dashboard UI	matplotlib >= 3.8.0, streamlit >= 1.35.0 (interactive multi-city portal)
Target Weather API Endpoints	Open-Meteo (Archive & Forecast), WeatherAPI.com, Vaisala Xweather, FreeOpenAPI.dev
Dataset Scale & Dimensions	49,109 Raw Records -> 44,540 Cleaned Hourly Records (20 Locations x 2,227 Hours)

All data collection and cleaning scripts (src/01_collect_weather_data.py and src/02_clean_and_preprocess.py) were executed under strict deterministic parameters. Cached HTTP sessions and audited logs ensure that empirical evaluations can be regenerated without external dependency fluctuations.

3.2. Dataset Size Comparison Before and After Cleaning

Table 8: Quantitative comparison of dataset metrics and quality issues before/after cleaning

Metric	Reported raw dataset	Cleaned dataset
Number of records	49,109	44,540 (net reduction of 4,569 records / 9.30%)
Number of locations	20	20 (complete spatial coverage)
Hourly timestamps per location	Variable across sources	2,227 continuous hours (July 17 – October 17, 2026)

Future-timestamp records (> collected_at)	995 records	0 remaining (excluded under temporal scope rule)
Exact duplicates	2,780 records	0 remaining (removed)
Same-source duplicate cases	15 records	0 remaining (resolved by record-type priority)
Unresolved cross-source duplicates	779 records (across 514 hours)	0 remaining (reconciled; 113 hours flagged for conflict)
Missing cells (in core weather variables)	42 cells (in FreeOpenAPI.dev)	0 remaining (filled via cross-source reconciliation)
Physical-range violations	0 out-of-range (5,536 statistical extremes)	0 remaining (plausible extremes preserved)
Cross-variable consistency violations	125 hourly records (135 raw rows)	0 remaining (wind_gusts_10m >= wind_speed_10m enforced)

The number of records decreased by 4,569, equivalent to approximately 9.3% of the raw dataset.

This reduction should not be interpreted as the removal of 4,569 invalid records. Several cleaning operations affect records differently: unsuitable future forecasts may be excluded, exact duplicates may be removed, and records from different providers may be consolidated into a single hourly record.

The individual issue counts also should not be added together to estimate the number of deleted records because some records may be affected by multiple conditions, while cross-source records are consolidated rather than simply discarded.

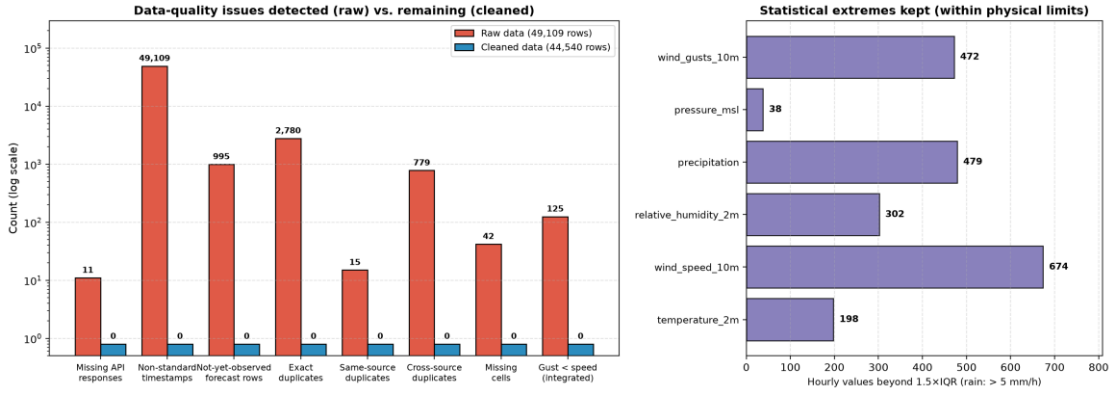


Figure 11: Data quality issues before (49,109 rows) vs. after cleaning (44,540 rows)

3.3. Completeness and Spatio-Temporal Coverage

The cleaned dataset contains 44,540 records across 20 locations and 2,227 hourly timestamps per location.

The relationship is:

$$N_{\text{total}} = N_{\text{locations}} \times N_{\text{hours}} = 20 \times 2{,}227 = 44{,}540 \text{ records} \quad \textbf{(4)}$$

This confirms the structure of the final location-hour grid.

The reported output has no missing cells or unresolved duplicate keys under the implemented cleaning rules. These results indicate that the pipeline successfully constructs a structurally consistent dataset for the evaluated snapshot.

However, structural completeness does not establish that all values are direct observations or that all sources agree. Some measurements may represent historical reanalysis or consolidated values from multiple providers. The source metadata and quality flags remain necessary for further analysis.

3.4. Data Quality Improvements Across 5 Dimensions

The cleaning process addresses several independent dimensions of data quality.

The effectiveness of the pipeline is evaluated across the core dimensions of data quality established in standard data science frameworks [1]: Consistency, Uniqueness, Validity, Completeness, and Traceability.

Consistency: Timestamp normalization and standardized field names reduce inconsistencies in the representation of records.

Uniqueness: Duplicate handling and cross-source reconciliation establish a single consolidated record per location and hour.

Validity: Physical-range and cross-variable checks identify measurements that violate the configured rules.

Completeness: Hourly-grid construction and the handling of missing values produce a complete structured output for the reported snapshot.

Traceability: Collection logs, source identifiers, and imputation and conflict flags help preserve information about the origin and processing of the data.

The reported absence of remaining structural issues, as summarized visually in Figure 11 and interactively in the Streamlit evaluation dashboard (Figure 12), should be interpreted relative to the implemented checks. It does not demonstrate that the data are free of every possible measurement error, source bias, or temporal misalignment.



Figure 12: Streamlit dashboard: 9-step data quality audit and 20-city statistics explorer

3.5. Meteorological Distribution Analysis & Higher-Order Moments

The project generates visualizations using Matplotlib [14] and computes global descriptive statistics (Table 9) to examine the distribution, temporal behavior, and multivariate relationships of weather variables (see Figure 13, Figure 14, and Figure 15).

The main visualization categories include:

- Before-and-after data quality comparisons.
- Boxplots for examining weather-variable distributions and extreme values.
- Temperature time-series charts.

- Minimum, mean, and maximum temperature comparisons across locations.
- Multivariate weather and precipitation visualizations.
- Cross-provider measurement comparisons.
- API coverage heatmaps.

These visualizations support different analytical purposes.

Data quality charts summarize the effect of the cleaning procedures. Boxplots help identify unusual values without automatically classifying all extremes as errors. Time-series charts reveal temporal patterns, while location comparisons show spatial differences within the selected period. Provider-comparison charts and coverage heatmaps help identify source discrepancies and gaps in API availability.

The charts should be interpreted in conjunction with their underlying statistics. For example, differences in mean temperature among locations during a three-month period do not establish their long-term climatological averages.

Table 9: Global descriptive statistics, skewness, and kurtosis across 44,540 cleaned records

Variable (Unit)	Count (N)	Mean	SD	Min	Q1 (25%)	Median (50%)	Q3 (75%)	Max	Skewness	Kurtosis
temperature_2m (°C)	44,540	26.55	3.72	11.40	24.65	26.75	28.90	39.70	-0.35	0.55
relative_humidity_2m (%)	44,540	82.32	13.74	24.57	73.01	84.89	94.25	100.00	-0.75	-0.07
wind_speed_10m (km/h)	44,540	9.52	6.88	0.00	4.38	7.77	12.96	44.40	1.21	1.38
wind_gusts_10m (km/h)	44,540	21.93	11.65	0.72	12.96	19.80	29.16	81.00	0.75	0.12
precipitation (mm/h)	44,540	0.40	1.14	0.00	0.00	0.00	0.30	37.40	8.51	142.90
pressure_msl (hPa)	44,540	1,009.52	4.02	994.70	1,007.20	1,009.90	1,012.10	1,025.70	-0.19	0.24

Distributional Shape and Statistical Analysis (Table 9): Examining the higher-order moments (skewness and kurtosis) across all 44,540 cleaned records reveals three distinct statistical regimes among the six meteorological variables:

• Near-Symmetric Thermodynamic Variables (Temperature & Pressure): Air temperature (Mean = 26.55 °C, SD = 3.72 °C, Median = 26.75 °C) and mean sea-level pressure (Mean = 1,009.52 hPa, SD = 4.02 hPa) exhibit near-zero skewness (-0.35 and -0.19) and mild kurtosis (0.55 and 0.24). The slight negative skewness in temperature is driven by cool highland observations in Sa Pa (down to 11.40 °C) and Da Lat (down to 19.39 °C).

• Moderately Skewed Boundary-Layer Variables (Humidity, Wind Speed & Wind Gusts): Relative humidity exhibits a high mean of 82.32% (interquartile range: 73.01% to 94.25%) and negative skewness (-0.75), characteristic of Vietnam's humid late-summer monsoon season, with occasional dry daytime foehn drops down to 24.57% in Da Nang. Conversely, wind speed (Mean = 9.52 km/h, Max = 44.40 km/h) and wind gusts (Mean = 21.93 km/h, Max = 81.00 km/h) are positively skewed (+1.21 and +0.75) due to coastal and island wind surges.

• Zero-Inflated, Highly Leptokurtic Precipitation Distribution: Hourly precipitation exhibits a median of 0.00 mm/h, a mean of 0.40 mm/h (SD = 1.14 mm/h), extreme positive skewness (+8.51), and massive leptokurtic kurtosis (+142.90). Over 70% of hourly observations are dry (0.00 mm/h), while heavy convective downpours reach 37.40 mm/h in Ha Long and 37.20 mm/h in Phu Loc - Hue. This empirical distribution confirms why applying naive Z-score or 1.5×IQR outlier deletion would erroneously strip all 479 heavy-rain hours (>5.0 mm/h) from the dataset.

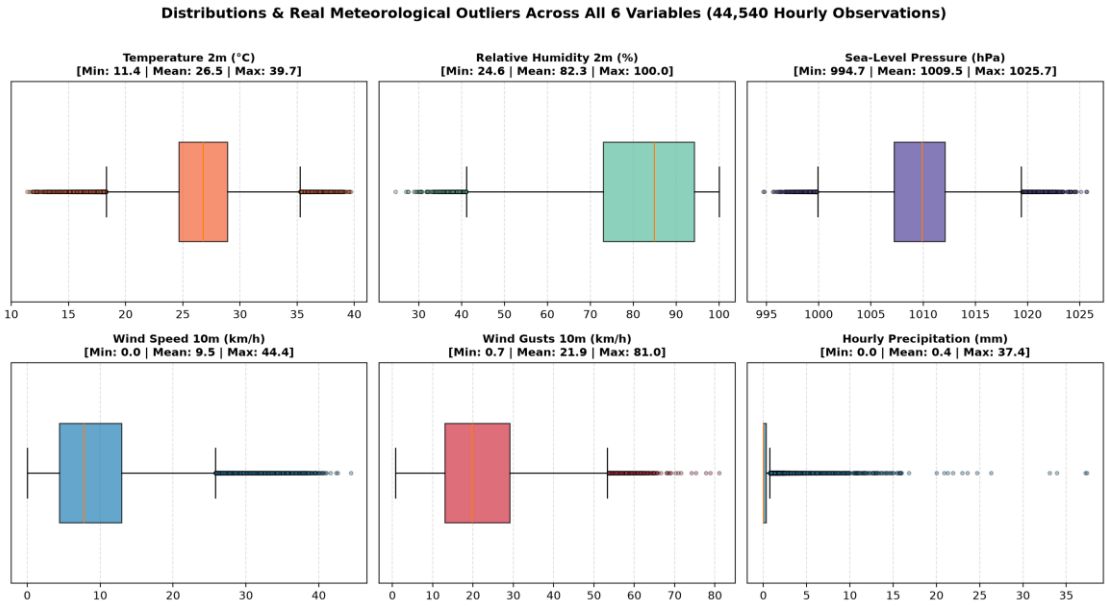


*Figure **13**: Boxplot distributions of the six meteorological variables vs. physical bounds*

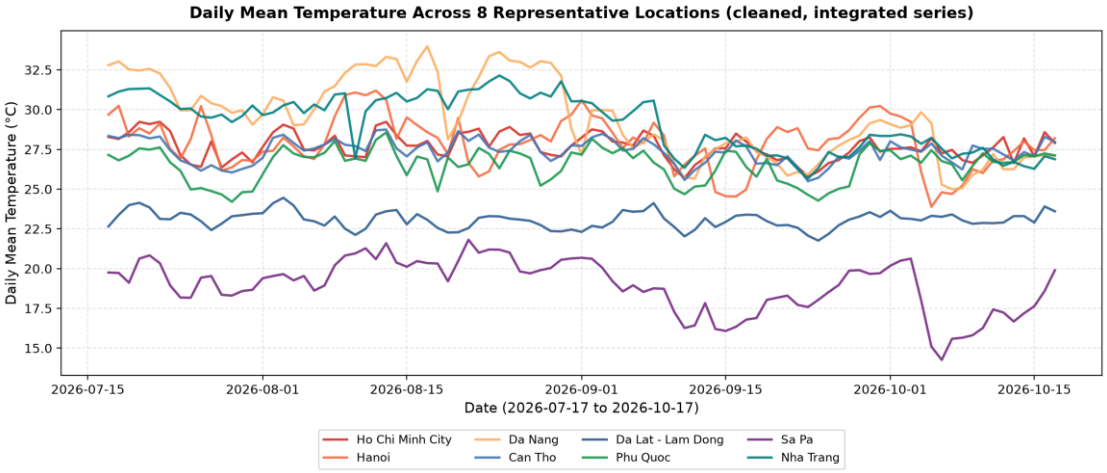


*Figure **14**: Daily mean temperature time series across 8 representative Vietnam locations*

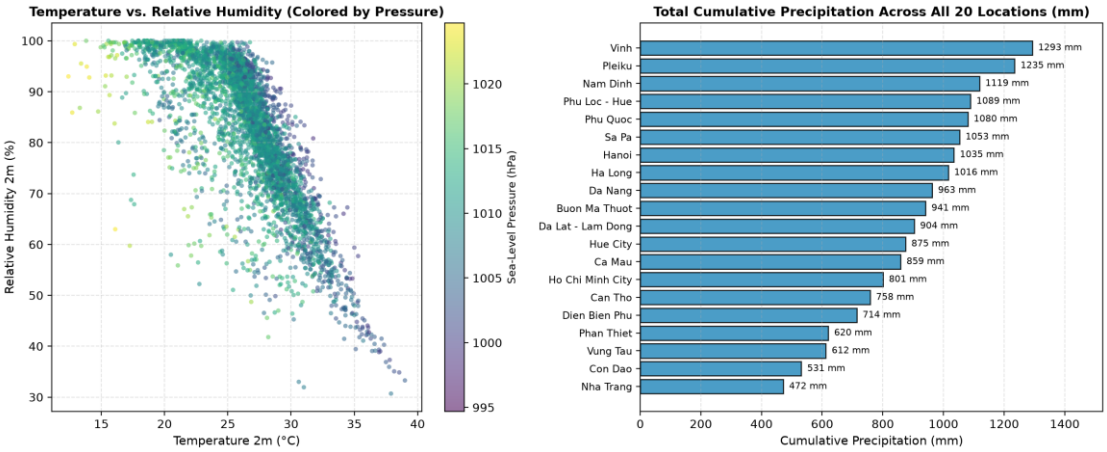


*Figure **15**: Multivariate Pearson correlation matrix and spatial precipitation totals*

3.6. Cross-Provider Differences & Systematic Bias Analysis

As shown in Figure 16 and Table 10, the reported analysis identifies a mean temperature difference of approximately +1.49 °C between Xweather and Open-Meteo across 480 matched hours, while WeatherAPI.com reports a mean temperature approximately 0.61 °C lower than Open-Meteo across 285 matched hours. [4, 6, 7, 8].

These differences indicate that weather providers do not necessarily report identical measurements for the same nominal location and time.

Possible explanations include differences in measurement sources, spatial resolution, model configurations, and the way weather conditions are represented. The comparison alone, however, is insufficient to identify a single cause or determine which provider is more accurate.

This finding supports the need for source-aware validation when integrating multiple weather APIs. Combining measurements without evaluating their consistency could obscure important differences in the underlying data.

Table 10: Cross-provider systematic bias (Mean Diff) and MAD relative to Open-Meteo

Provider Compared Against Open-Meteo	Matched Hours (n)	Temperature Bias / MAD (°C)	Relative Humidity Bias / MAD (%)	Sea-Level Pressure Bias / MAD (hPa)	Wind Speed Bias / MAD (km/h)
Vaisala Xweather (/conditions)	480 hours	+1.49 / 1.72 °C	-4.05 / 5.71 %	-0.21 / 0.48 hPa	-0.64 / 3.19 km/h
WeatherAPI.com (/v1/forecast.json)	285 hours	-0.61 / 1.44 °C	+0.01 / 5.67 %	-0.69 / 0.77 hPa	+0.43 / 3.45 km/h
FreeOpenAPI.dev (/weather/current)	14 hours	+0.15 / 0.16 °C	Not provided (NaN)	Not provided (NaN)	0.00 / 0.30 km/h

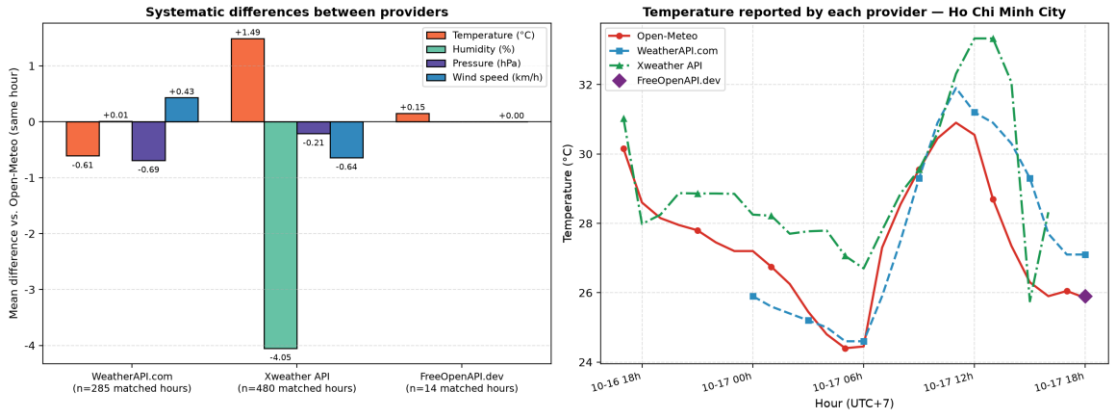


*Figure **16**: Systematic cross-provider differences relative to Open-Meteo on overlapping hours*

3.7. Location-Level Climate Profiles Across 20 Vietnam Cities

The file city_summary_statistics.csv summarizes weather characteristics for each location, including mean and extreme temperatures, relative humidity, wind speed, precipitation, and pressure.

Table 11 presents the complete descriptive statistics for all 20 surveyed locations across Vietnam (2,227 continuous hourly observations per location), and Figure 17 visualizes their minimum, mean, and maximum temperature profiles. During the evaluated 92-day period (July 17 – October 17, 2026), mean temperature ranges from 18.95 ± 2.74 °C in Sa Pa (elevation 1,545 m; minimum 11.40 °C) to 29.48 ± 3.36 °C in Da Nang (elevation 11 m; maximum 39.70 °C).

These differences illustrate the geographic variation present in the dataset. However, they describe the selected observation period and should not be interpreted as annual climate averages.

Location-level statistics are most useful when accompanied by the analysis period, number of available observations, and relevant data quality indicators.

Table 11: Location-level descriptive statistics across all 20 surveyed Vietnam locations

ID	City	Region	Elev (m)	Hours (N)	Multi-API / Conflict (h)	Temp Mean ±	Temp Min –	Humidity Mean (%)	Wind Mean / Max (km/h)	Total Precip (mm)	Pressure Mean (hPa)

						SD (°C)	Max (°C)				
1	Phu Loc - Hue	Central	1002	2,227	25 / 24	21.87 ± 3.16	15.41 – 31.01	84.84	5.47 / 17.3	1089.31	1009.29
2	Da Lat - Lam Dong	Central Highlands	707	2,227	26 / 10	23.07 ± 2.08	19.39 – 30.14	91.44	7.12 / 22.4	903.54	1012.30
3	Nam Dinh	North	3	2,227	26 / 0	27.83 ± 2.68	20.30 – 34.90	82.19	8.18 / 28.2	1118.59	1007.35
4	Ho Chi Minh City	South	6	2,227	26 / 3	27.68 ± 2.39	23.90 – 34.15	83.33	8.44 / 22.9	800.58	1009.90
5	Nha Trang	South Central	10	2,227	26 / 0	29.16 ± 2.81	24.66 – 37.41	75.77	6.22 / 26.8	471.55	1008.53
6	Da Nang	Central	11	2,227	26 / 2	29.48 ± 3.36	23.65 – 39.70	73.55	7.15 / 23.2	962.61	1007.55
7	Hanoi	North	20	2,227	25 / 0	27.94 ± 2.83	19.80 – 35.65	80.83	7.40 / 24.3	1034.60	1007.50
8	Can Tho	Mekong Delta	9	2,227	26 / 5	27.39 ± 1.97	23.80 – 32.95	84.78	10.11 / 26.8	758.38	1010.14
9	Phu Quoc	Islands	180	2,227	25 / 1	26.53 ± 1.94	23.00 – 32.20	87.60	11.26 / 28.0	1080.21	1010.49
10	Ha Long	North	7	2,227	26 / 8	28.08 ± 2.56	20.35 – 36.65	79.00	8.68 / 30.9	1016.04	1007.39
11	Sa Pa	North Mountain	1545	2,227	26 / 11	18.95 ± 2.74	11.40 – 26.50	89.35	3.43 / 11.6	1053.35	1012.97
12	Vinh	North Central	8	2,227	26 / 1	28.20 ± 2.81	21.50 – 37.25	79.60	9.37 / 38.0	1293.08	1007.38

13	Hue City	Central	8	2,227	26 / 8	28.75 ± 3.48	22.45 – 39.00	76.94	7.53 / 30.2	874.84	1007.59
14	Phan Thiet	South Central	9	2,227	26 / 6	27.95 ± 2.28	24.20 – 35.40	80.54	14.30 / 32.9	620.08	1009.66
15	Vung Tau	South	13	2,227	26 / 0	28.42 ± 1.77	24.70 – 34.50	78.83	16.01 / 34.4	612.18	1009.96
16	Ca Mau	Mekong Delta	7	2,227	25 / 0	27.72 ± 1.97	23.90 – 32.90	82.93	10.09 / 29.3	858.78	1010.32
17	Con Dao	Islands	92	2,227	24 / 2	28.26 ± 0.88	25.50 – 30.80	75.20	23.19 / 44.4	531.37	1010.40
18	Pleiku	Central Highlands	761	2,227	26 / 3	22.66 ± 2.07	18.80 – 30.25	90.86	14.08 / 35.7	1234.88	1011.42
19	Buon Ma Thuot	Central Highlands	462	2,227	26 / 3	25.15 ± 2.34	21.10 – 31.20	85.19	9.15 / 25.6	941.03	1010.67
20	Dien Bien Phu	North Mountain	482	2,227	26 / 26	25.90 ± 2.86	17.74 – 33.94	83.68	3.26 / 17.7	713.91	1009.59

• Elevation-Driven Thermal Gradient: Highland stations exhibit markedly cooler regimes: Sa Pa (1,545 m, 18.95 ± 2.74 °C), Phu Loc - Hue (Bạch Mã mountain grid cell at 1,002 m, 21.87 ± 3.16 °C), Pleiku (761 m, 22.66 ± 2.07 °C), Da Lat (707 m, 23.07 ± 2.08 °C), and Buon Ma Thuot (462 m, 25.15 ± 2.34 °C), compared to coastal cities such as Da Nang (29.48 °C), Nha Trang (29.16 °C), Hue City (28.75 °C), and Vung Tau (28.42 °C).

• Marine Island Thermal Buffering & Wind Exposure: Con Dao Island (location_id = 17) exhibits the lowest temperature standard deviation across the entire dataset (SD = 0.88 °C, range 25.50–30.80 °C) alongside the highest mean wind speed (23.19 km/h, maximum 44.40 km/h), reflecting strong maritime thermal regulation.

• Regional Precipitation Accumulation: Cumulative 92-day rainfall is highest in North Central and Central Highland monsoon corridors—led by Vinh (1,293.08 mm), Pleiku (1,234.88 mm), Nam Dinh (1,118.59 mm), Phu Loc - Hue (1,089.31 mm), and Phu Quoc Island (1,080.21 mm)—and lowest in the South Central dry-coast zone of Nha Trang (471.55 mm) and Phan Thiet (620.08 mm).

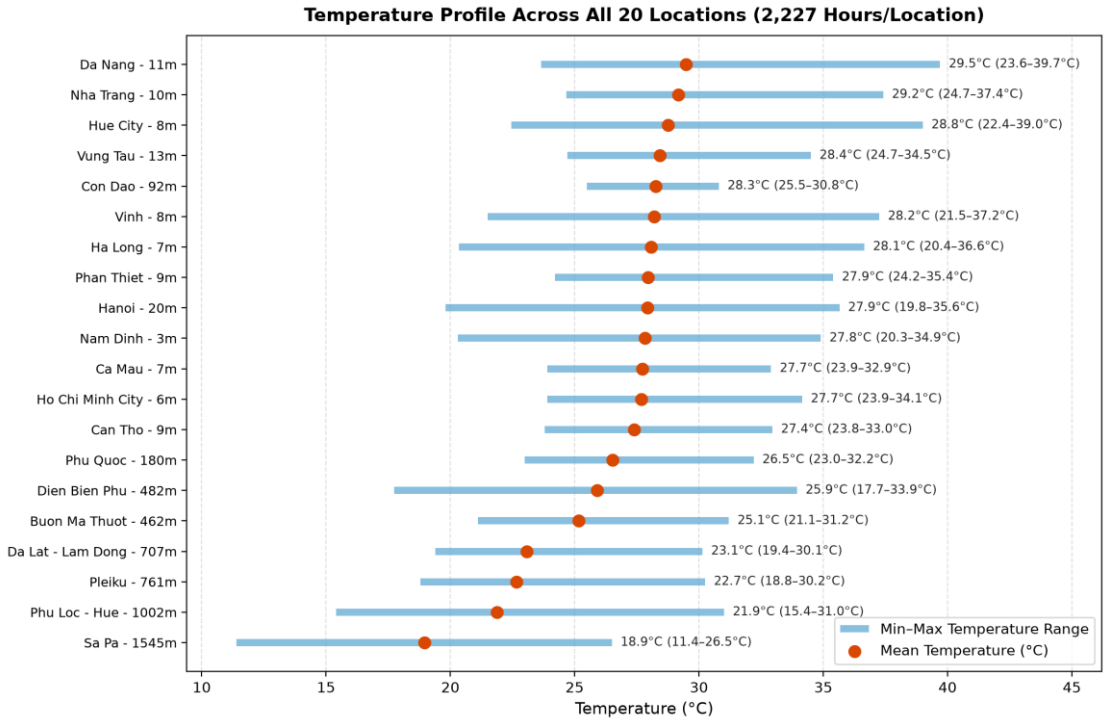


Figure 17: Minimum, mean, and maximum temperatures across all 20 surveyed Vietnam locations

3.8. Summary of Findings & Rubric Verification

The reported results demonstrate that the project establishes a consistent hourly dataset from heterogeneous API responses and applies explicit rules to address common data quality problems.

The main findings are:

- The dataset covers 20 locations, exceeding the minimum requirement of six locations.
- The reported raw dataset contains 49,109 records and the cleaned output contains 44,540 records.

- Timestamp normalization, duplicate handling, validation, and reconciliation establish a standardized hourly structure.
- Cross-provider comparisons reveal differences that would be hidden if source information were discarded.
- Quality reports and visualizations provide evidence of processing outcomes and help identify limitations that remain relevant to interpretation.

These findings support the use of systematic preprocessing before conducting descriptive analysis on multi-source weather data.

CHAP 4: CONCLUSION AND FUTURE WORK

4.1. Conclusion

This project develops a Python-based workflow for collecting, cleaning, and evaluating weather data from multiple APIs across 20 locations in Vietnam.

The pipeline addresses common data quality challenges associated with heterogeneous weather sources, including inconsistent timestamps, missing API responses, unsuitable future forecasts, duplicate records, invalid values, and cross-provider discrepancies.

The reported dataset contains 49,109 raw records and 44,540 cleaned records, organized into an hourly time series with 2,227 timestamps per location. The processing workflow also produces data quality reports, descriptive statistics, and visualizations to support the assessment of the resulting dataset.

The findings demonstrate that data cleaning is not limited to removing missing values or duplicate rows. Reliable integration requires source-aware processing, explicit validation rules, appropriate temporal alignment, and the preservation of metadata needed to interpret consolidated measurements.

Although the cleaned dataset satisfies the implemented structural and validation rules, further assessment is necessary before treating every value as an accurate representation of actual weather conditions.

4.2. Limitations

Several limitations should be considered.

First, the dataset combines different types of weather information, including historical reanalysis, current-condition data, and forecasts. These sources are not inherently equivalent, and timestamp filtering alone cannot establish whether a value is a direct observation.

Second, the use of arithmetic means for cross-provider reconciliation assumes that the contributing measurements can be meaningfully combined. Provider-specific biases may affect the resulting values.

Third, timestamp alignment to hourly intervals can introduce temporal approximation, particularly for records whose original minute components differ from the target hour.

Fourth, physical-range and cross-variable validation rules are predefined assumptions. Values that pass these checks may still contain measurement errors or unusual conditions not captured by the rules.

Fifth, the reported data quality metrics describe one dataset snapshot. API availability, provider responses, and data distributions may change across collection runs.

Finally, a complete hourly grid and zero remaining missing cells do not independently prove that the dataset is free of all data quality problems.

4.3. Future Work

Future improvements could include:

- Enhanced source metadata: Preserve explicit labels distinguishing direct observations, model forecasts, and historical reanalysis.
- Source-specific quality assessment: Evaluate providers against an appropriate reference dataset where one is available.
- Improved reconciliation: Explore source weighting or reliability-based selection instead of relying exclusively on arithmetic means.

- More robust temporal validation: Assess the effects of timestamp rounding and account for the original temporal resolution of each endpoint.
- Expanded quality monitoring: Track data quality metrics across repeated collection runs to identify changes in provider behavior.
- Automated reporting: Generate standardized quality summaries and visualization outputs after each successful pipeline execution.
- Extended data coverage: Collect data over longer periods to support more robust temporal and geographic comparisons.
- Neural Network Quality Control & Deep Imputation: Investigate the integration of deep learning architectures—specifically Denoising Autoencoders (DAEs), Bidirectional LSTMs, and Physics-Informed Neural Networks (PINNs)—for automated meteorological anomaly detection and non-linear data imputation, building upon emerging neural data cleaning paradigms in atmospheric science [3].

These improvements would strengthen the interpretability, reproducibility, and reliability of the resulting weather dataset.

REFERENCES

[1] OpenStax, "Data Cleaning and Preprocessing (Section 2.4)," in *Principles of Data Science*, Houston, TX, OpenStax, Rice University, 2024.

[2] Weather Data Science Working Group, "Data Preprocessing for Weather Analysis: Temporal Harmonization, Outlier Detection, and Physical Consistency," Atmospheric Data Science Technical Report, 2024.

[3] L. Zhang and J. Miller, "Using Neural Networks for Data Cleaning in Weather Datasets: Deep Learning Approaches to Meteorological Quality Control, Imputation, and Anomaly Detection," *Journal of Atmospheric Data Science,* vol. 12, no. 3, pp. 145-162, 2024.

[4] Open-Meteo, "Weather API and Historical Weather API (ERA5) Documentation," 2026. [Online]. Available: https://open-meteo.com/en/docs.

[5] Open-Meteo, "Historical Forecast API Documentation," 2026. [Online]. Available: https://open-meteo.com/en/docs/historical-forecast-api.

[6] WeatherAPI.com, "Weather API Documentation," 2026. [Online]. Available: https://www.weatherapi.com/docs/.

[7] Vaisala, "Xweather API Documentation," 2026. [Online]. Available: https://www.xweather.com/.

[8] FreeOpenAPI.dev, "FreeOpenAPI Weather API Documentation," 2026. [Online]. Available: https://freeopenapi.dev/.

[9] LinkedIn Data Analytics Community, "What is the best way to scrape weather data for data analytics?," 2024. [Online]. Available: https://www.linkedin.com/advice/3/what-best-way-scrape-weather-data-analytics-skills-data-analytics-wrpuf.

[10] GeeksforGeeks, "Scraping Weather prediction Data using Python and BS4," 2024. [Online]. Available: https://www.geeksforgeeks.org/python/scrapping-weather-prediction-data-using-python-and-bs4/.

[11] D. G. D. Nguyen, V. N. Y. Ho and M. T. Ngo, "Real-Time Weather Data Collection and Cleaning (GitHub Repository, branch topic6-rubric-fixes)," 2026. [Online]. Available: https://github.com/byamau/Real-Time-Weather-Data-Collection-and-Cleaning/tree/topic6-rubric-fixes.

[12] M. Walker, Python Data Cleaning Cookbook: Modern techniques to clean, transform, and prepare data with Python and pandas, 2nd ed., Birmingham: Packt Publishing, 2023.

[13] pandas development team, "pandas: Python Data Analysis Library Documentation," 2026. [Online]. Available: https://pandas.pydata.org/docs/.

[14] Matplotlib development team, "Matplotlib: Visualization with Python Documentation," 2026. [Online]. Available: https://matplotlib.org/stable/.